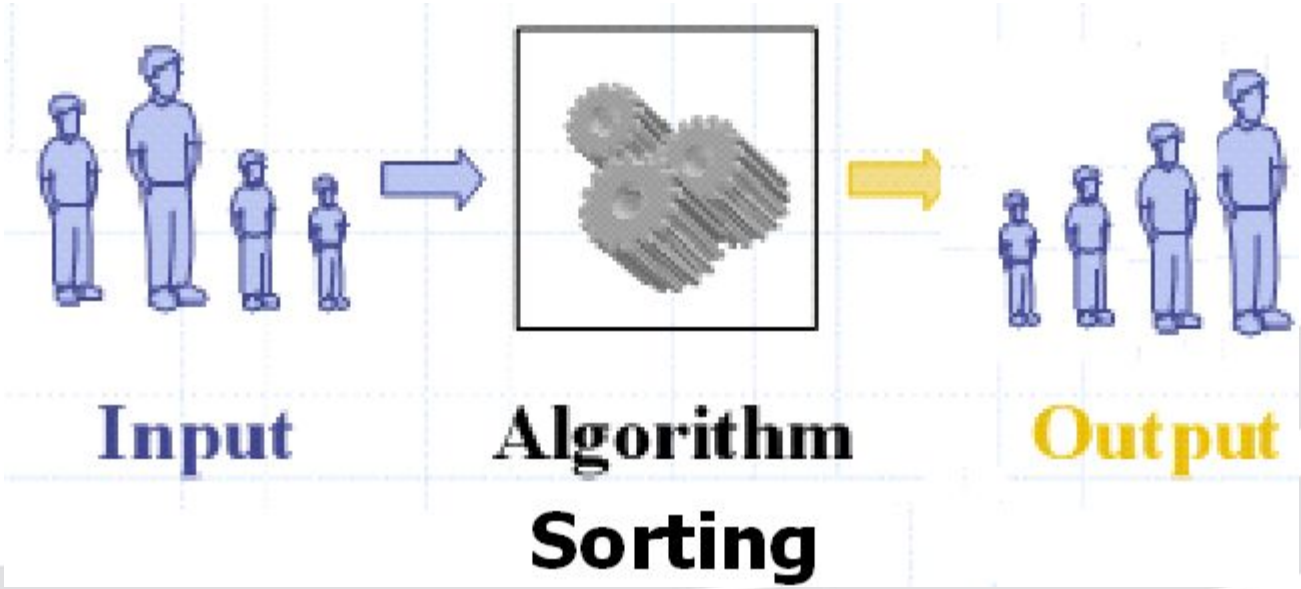
Bài 10.
Sắp xếp (Sorting)



Nội dung

I. Bài toán sắp xếp
II. Sắp xếp nổi bọt (Bubble sort)
III. Sắp xếp chọn (Selection sort)
IV. Sắp xếp chèn (Insertiong sort)
V. Bài tập

I. Bài toán

- **Input:**
  - Dãy các phần tử (và một thứ tự)

  (Dãy các phần tử thường được lưu bằng mảng.)
- **Output:**
  - Dãy các phần tử được sắp xếp theo thứ tự tăng hoặc giảm dần theo một hoặc một vài thuộc tính của nó (các thuộc tính này gọi là thuộc tính khóa).
- **Thuộc tính khóa được sắp xếp theo một hàm logic, ví dụ (<=) hoặc các toán tử so sánh khác.**

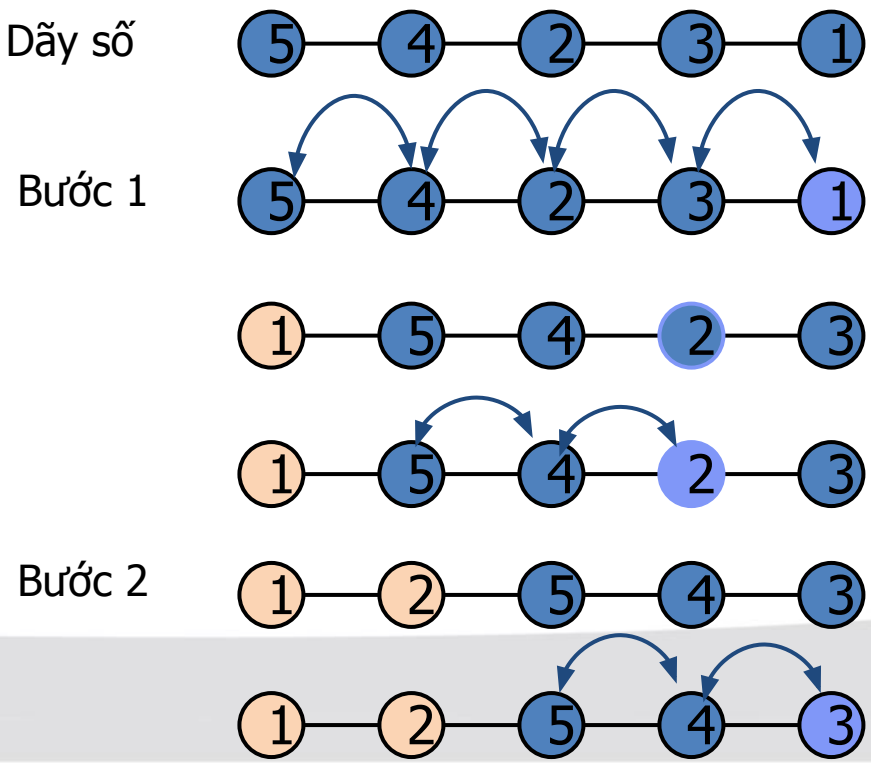
Các thuật toán với thời gian chạy $O(n^2)$

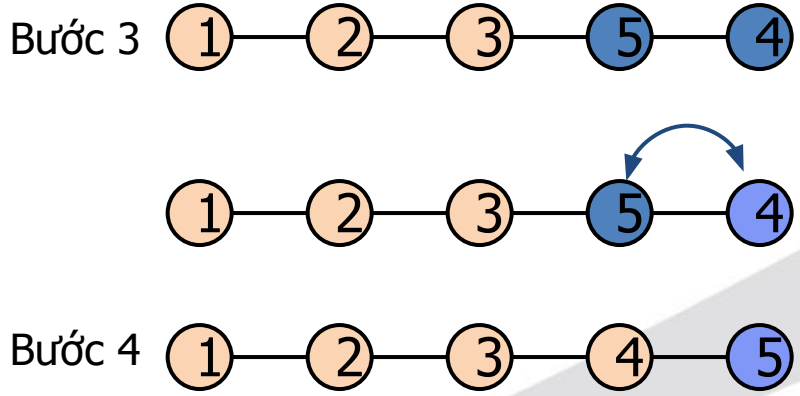
- **Nổi bọt – Bubble sort**
- **Chèn – Insertion sort**
- **Chọn – Selection sort**

II. Sắp xếp nổi bọt – Bubble sort

a. Ý tưởng: **Thực hiện chuyển dần các phân tử có giá trị khóa nhỏ về đầu dãy, các phần tử có khóa lớn về cuối dãy.**

Ví dụ: sắp xếp dãy sau theo thứ tự tăng dần:





b.Thuật toán

Algorithm ***BubbleSort(Array A, n)***
Input: Mảng A có n phần tử
Output: Mảng A được sắp theo thứ tự tăng dần của khóa

```
for i ← 0 to n-2 do
    for j ← n-1 downto i+1 do
     if A[j].Key < A[j-1].Key then
          swap(A[j-1], A[j]);
```

- Trong đó swap là thủ tục tráo đổi vị trí của hai phần tử

```
void Swap(object &a, object &b){
    Object tg;
    tg = a;  a = b;   b = tg;
}
```

Chứng minh thời gian chạy của thuật toán trong trường hợp xấu nhất là $O(n^2)$

?

c. Thời gian chạy

```
Algorithm BubbleSort(Array A, n)
Input: Mảng A có n phần tử
Output: Mảng A được sắp theo thứ tự tăng dần của khóa
    for i ← 0 to n-2 do                    n+2
        for j ← n-1 downto i+1 do                  n-i+3
         if A[j].Key < A[j-1].Key then        4
             swap(A[j-1], A[j]);              6
```

- Thời gian chạy:

$$T(n) = (n+2) + (n-1)*3 + 10*[(n-1) + (n-2) + ..2+1]$$

- Thời gian chạy của thuật toán là $O(n^2)$

Ví dụ:

Mô tả quá trình sắp xếp của dãy số

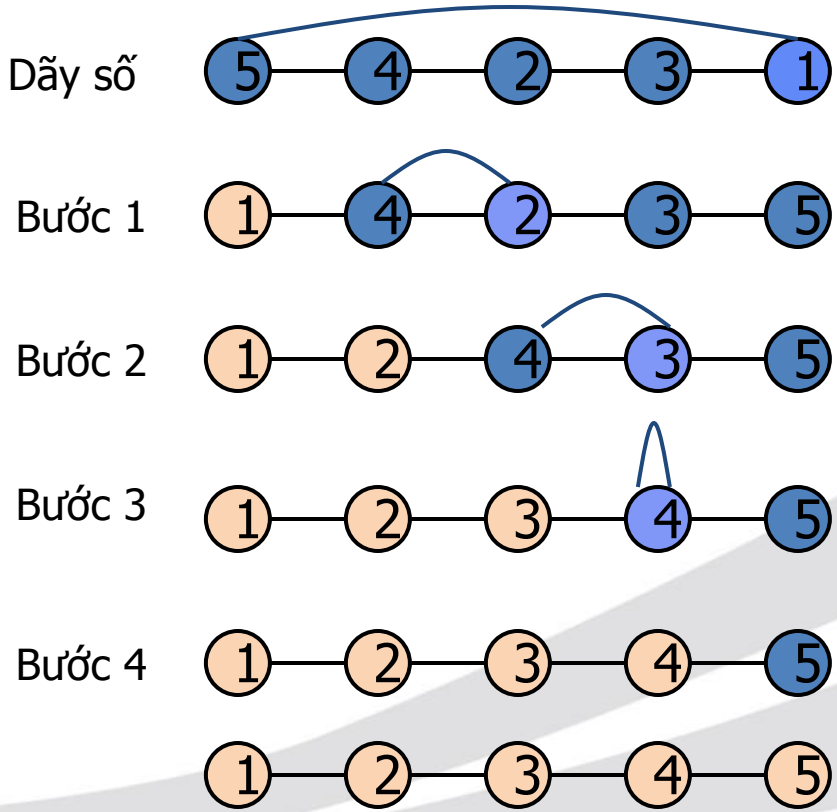
12 43 11 34 23 43

III. Sắp xếp chọn - Selection sort

a. Ý tưởng: Chọn phần tử có khóa nhỏ nhất trong các phần tử còn lại chuyển nó về đầu và loại bỏ nó khỏi dãy.

- **Ví dụ** sắp xếp dãy sau theo thứ tự tăng dần:





b. Thuật toán

Algorithm ***SelectionSort(Array A, n)***
Input: Mảng A có n phần tử
Output: Mảng A được sắp theo thứ tự tăng dần của khóa

```
for i ← 0 to n-2  do
    posmin ← i ;
        for j ← i+1 to n-1 do
       if A[posmin].Key > A[j].Key then
        posmin ← j ;
    if posmin ≠ i then
            swap (A[i], A[posmin]);
```

Chứng minh thời gian chạy của thuật toán trong trường hợp xấu nhất là $O(n^2)$

?

c. Thời gian chạy

```
for i ← 0 to n-2  do                                    n+2
    posmin ← i ;                                        n-1
        for j ← i+1 to n-1 do                    n-i + 3(n-1)
       if A[posmin].Key > A[j].Key then          3(n-i)
          posmin ← j ;                           n-i
    if posmin ≠ i then                           n-1
            swap (A[i], A[posmin]);              5(n-1)
```

Thời gian chạy của thuật toán

$$T(n) = (n+2) + 5*[(n-1)+(n-2)+..+1] + 10*(n-1)$$

Thời gian chạy của thuật toán là $O(n^2)$

Ví dụ:

Mô tả quá trình sắp xếp của dãy số

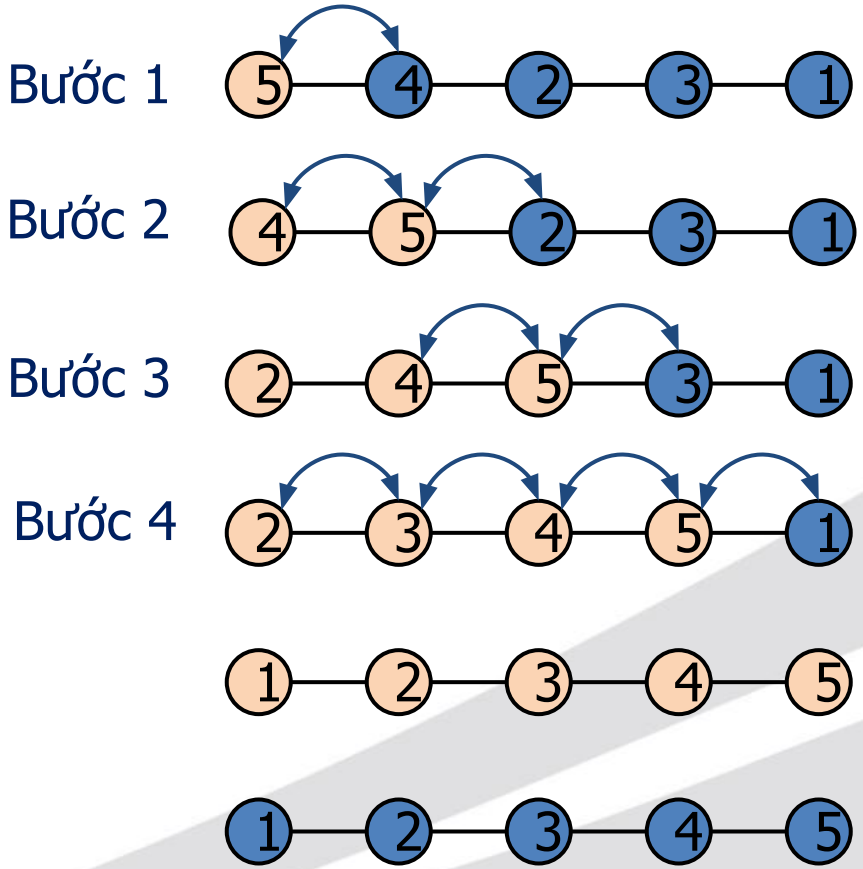
12 43 11 34 23 435

IV. Sắp xếp chèn – Insertion sort

a. Ý tưởng: Lấy phần tử thứ A[j] chèn vào dãy gồm các phần tử từ A[1]..A[j-1] sao cho ta được dãy A[1]..A[j] được sắp. Trong đó dãy A[1]..A[j-1] là dãy đã được sắp.

- **Ví dụ:** sắp xếp dãy sau theo thứ tự tăng dần:





b. Thuật toán

Algorithm ***InsertionSort(Array A, n)***
Input: Mảng A có n phần tử
Output: Mảng A được sắp theo thứ tự tăng dần của khóa

```
for i ← 1 to n-1 do
    j ← i-1;
    x ← A[i];
    while (A[j].Key>x.Key) and (j>=0) do
        A[j+1] ← A[j];
        j ← j-1;
    A[j+1] ← x;
```

Chứng minh thời gian chạy của thuật toán trong trường hợp xấu nhất là $O(n^2)$

?

Ví dụ:

Mô tả quá trình sắp xếp của dãy số

12 43 11 34 23 43

V. Bài tập

1. Định nghĩa lớp sinh viên có các thuộc tính: Mã SV, Ho đệm, tên, khóa, lớp.
2. Nhập vào 1 danh sách sinh viên
3. Sắp xếp sinh viên theo thứ tự alphabet của tên.
4. In danh sách sinh viên lên màn hình.